CE 4011
Special Topics in Civil Engineering
Structural Analysis Software Development

Assignment #4
Software Implementation Report

Structural FEA Solver
Feature Extensions

Student Information

Name: **Mohammad Umair Naeem**
Student ID: **2416055**

Term: Spring Semester, 2025–2026

Department of Civil Engineering

Contents

1 Introduction

This report documents the design, implementation, and validation of the Assignment #4 structural FEA solver extension, developed for CE 4011 — Structural Analysis Software Development. The primary contributions of this assignment are the addition of two advanced loading capabilities to the existing 2D frame/truss solver: (1) thermal loading, encompassing both uniform temperature changes and linear thermal gradients across cross-sections, and (2) support settlement analysis, enabling prescribed nodal displacements to be incorporated as kinematic boundary conditions in the global system. The implementation follows an object-oriented design philosophy, introducing a new `TemperatureL` load class, an extended `Support` dataclass, and a matrix partitioning solver strategy. All results have been rigorously validated against the industry-standard SAP2000 FEA software for two benchmark problems (Q2a: settlement case, Q2b: thermal loading case). A comprehensive testing suite — including unit tests, interface tests, and backward-compatibility regression tests — confirms correctness and reliability.

All assignment source code, XML input files, and test scripts are publicly available at the following GitHub repository:

https://github.com/umairnaeem2703/CE4011-Assignment-4-Submission

2 Revisions to the Core Architecture

2.1 Motivation: Static XML to Dynamic Class Structure

The foundation of the Assignment #4 enhancements lies in a fundamental architectural transition from purely static XML input definitions to a robust, extensible object-oriented data model. The original implementation relied on fixed XML attributes to encode Fixed-End Force (FEF) conditions directly, which limited expressivity and scalability.

2.2 The FEF Decoupling Strategy

2.2.1 Problem with Static Attributes

In the previous design, thermal and settlement effects were scattered across rigid XML schemas. Fixed-End Forces (FEF) were computed as static, immutable properties of elements—fixed at parse time with no dynamic reconciliation or extension mechanism.

2.2.2 Solution: Specialized `TemperatureL` Class

To address this, the refactored architecture introduces a specialized `TemperatureL` class that inherits from the abstract `MemberLoad` base class. This decoupling brings several critical advantages:

- **Single Responsibility:** Thermal FEF logic is isolated in a dedicated class.
- **Extensibility:** New load types (e.g., soil pressure, humidity effects) can be added without modifying core element physics.
- **Testability:** Thermal calculations are independently verifiable.
- **Mathematical Clarity:** The physics is explicitly codified in the `FEF()` method.

2.3 Enhanced XML Parser with Expressive Tags

The new XML schema supports richer semantics through dedicated `<support>` and `<load>` tags:

- **Support Element:** Encapsulates both restraint flags (`restrain_ux`, `restrain_uy`, `restrain_rz`) and prescribed settlement values (`settlement_ux`, `settlement_uy`, `settlement_rz`).
- **Load Element:** Polymorphically instantiates the appropriate load class (`NodalLoad`, `PointLoad`, `UniformlyDL`, `TemperatureL`).

This design enables the parser to dynamically construct the correct object graph at runtime, respecting the inheritance hierarchy and enabling polymorphic load assembly.

2.4 Domain Model Class Hierarchy

The domain model now cleanly separates:

1. **Geometric Entities:** Material, Section, Node, Element
2. **Boundary Conditions:** Support (with settlement extension)
3. **Load Hierarchy:** Abstract Load class with concrete implementations (NodalLoad, PointLoad, UniformlyDL, TemperatureL)
4. **Container Structures:** LoadCase, StructuralModel

This modular design directly supports the advanced features required in Assignment #4: thermal loading and settlement-induced forces.

3 Object-Oriented Design

This section documents the complete UML architecture of the developed solver. The design employs standard object-oriented patterns including inheritance (Load hierarchy), composition (StructuralModel aggregates all entities), and polymorphism (FEF computation via abstract Load interface).

3.1 Class Diagram: Domain Model

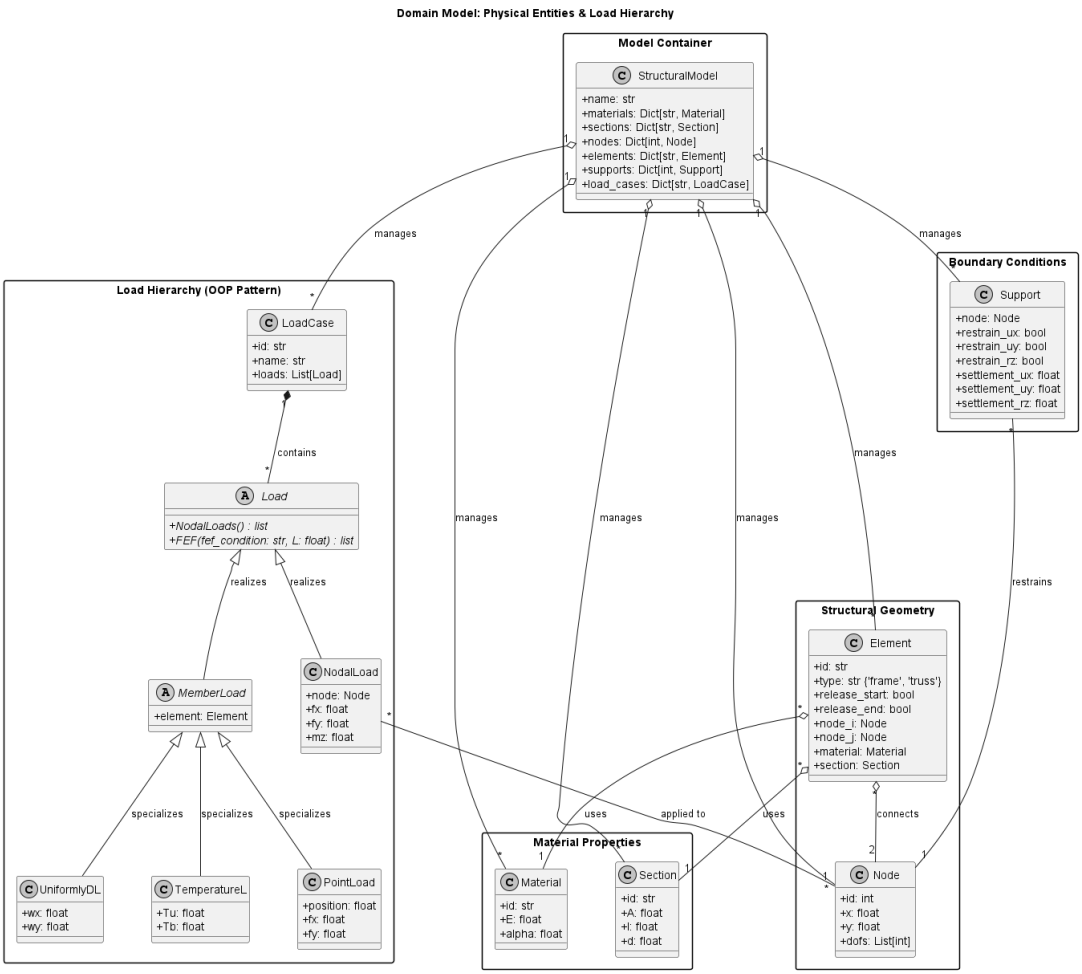

Figure 1: Domain Model UML Class Diagram. Shows the structural hierarchy: Material, Section, Node, Element form the geometric basis. Support and Load subclasses define boundary conditions and applied forces. LoadCase and StructuralModel provide container semantics.

3.2 Class Diagram: Analysis Logic

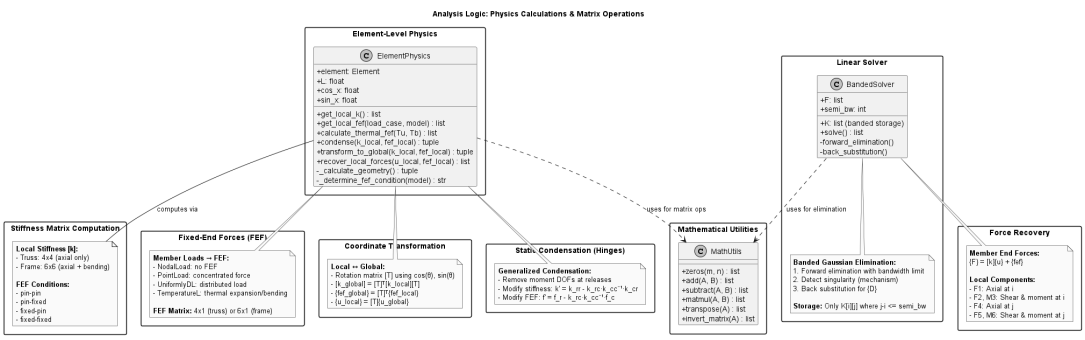

Figure 2: Analysis Logic UML Diagram. Depicts the solver pipeline: XMLParser constructs the StructuralModel, GlobalAssembler builds the global stiffness matrix (K) and load vector (F), Solver solves $\mathbf{KD} = \mathbf{F}$, and PostProcessor extracts and formats results.

3.3 Class Diagram: System Architecture

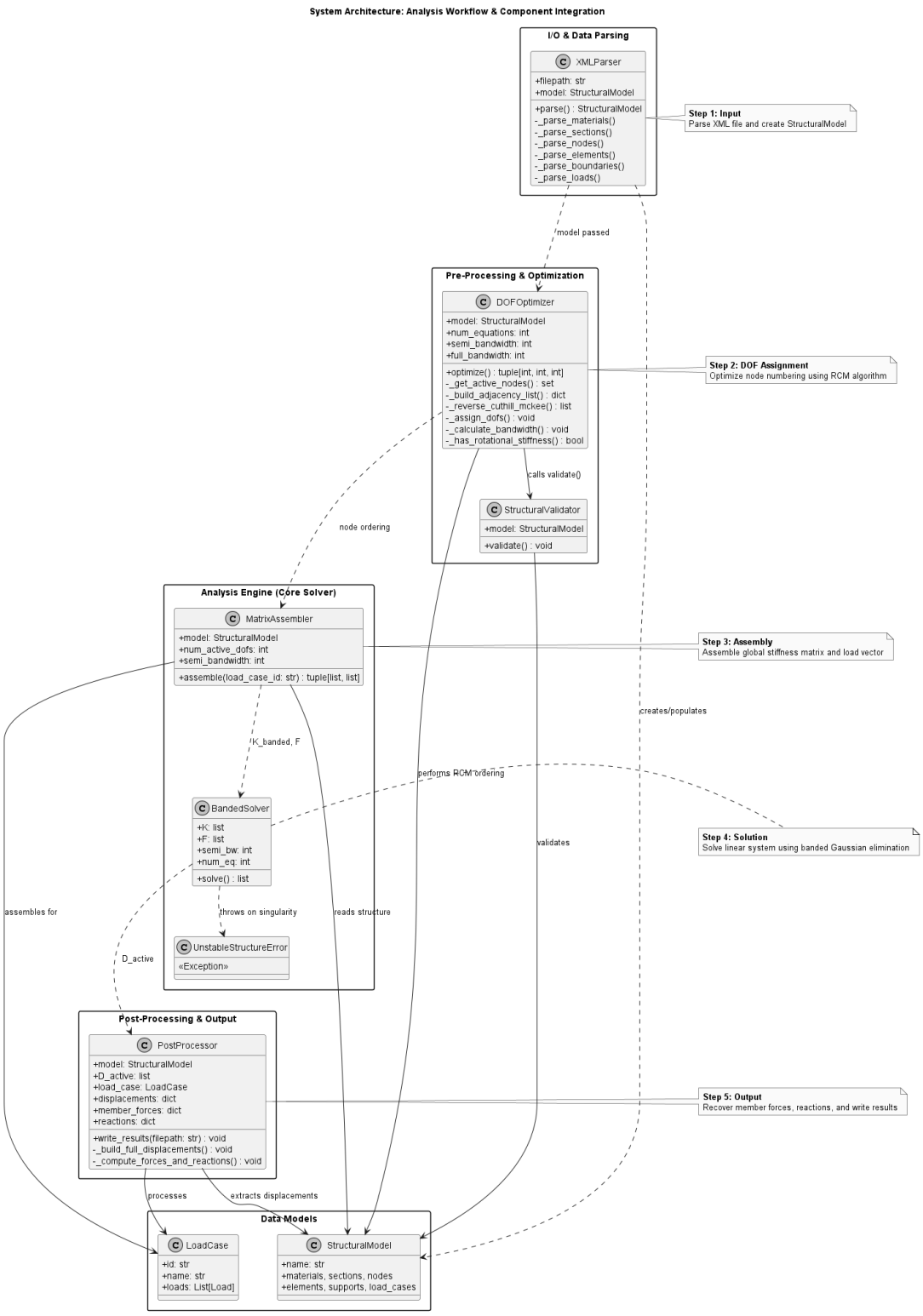

Figure 3: System Architecture Diagram. Illustrates the full software stack with module dependencies: parser.py defines data models, element_physics.py computes local stiffness and FEF, matrix_assembly.py assembles global matrices, banded_solver.py solves the linear system, and post_processor.py extracts results.

3.4 Activity Diagram: Solver Workflow

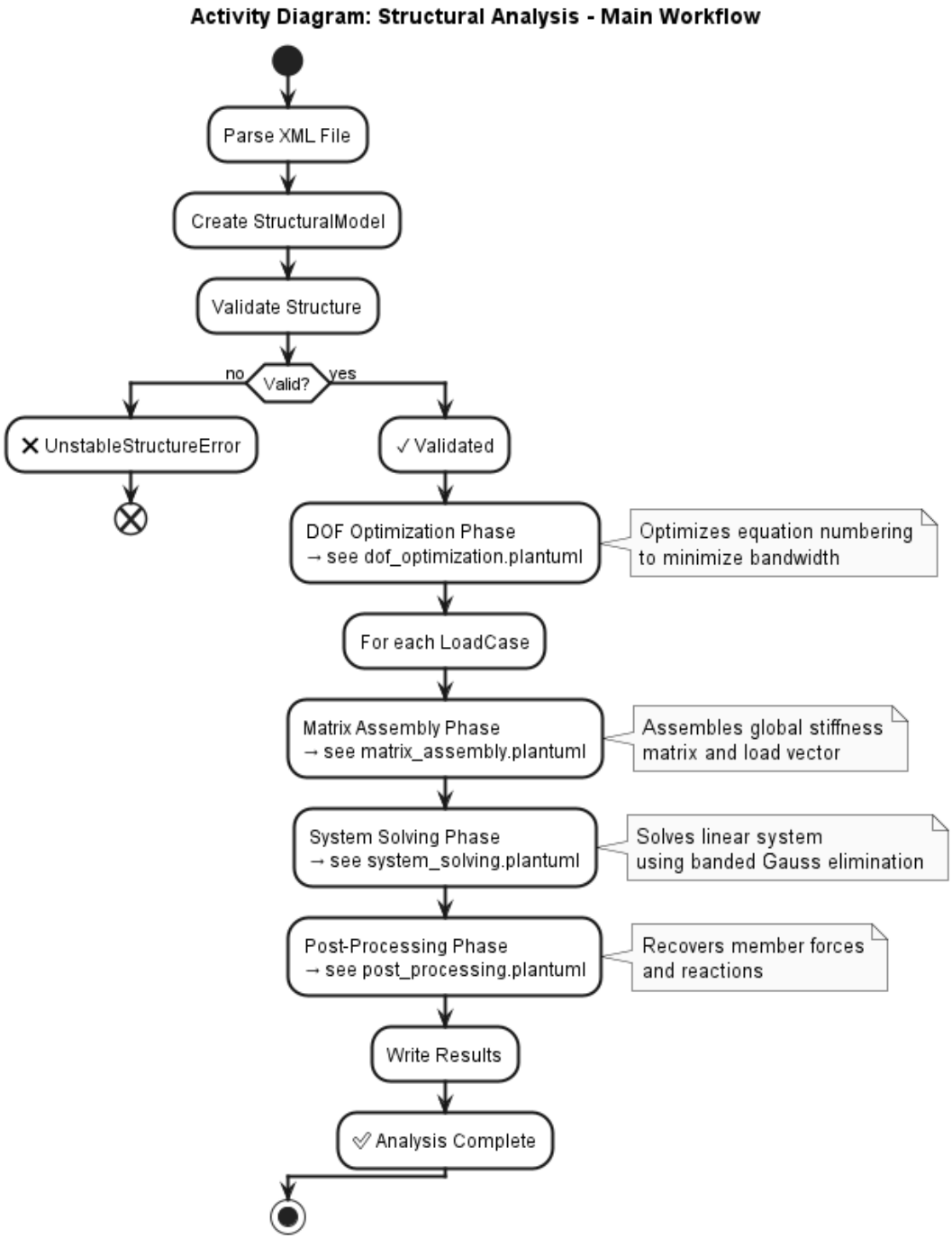

Figure 4: Activity Diagram showing the high-level control flow: Parse XML → Assemble global matrices → Apply boundary conditions → Solve linear system → Post-process and output results.

3.5 Sequence Diagram: Interaction Protocol

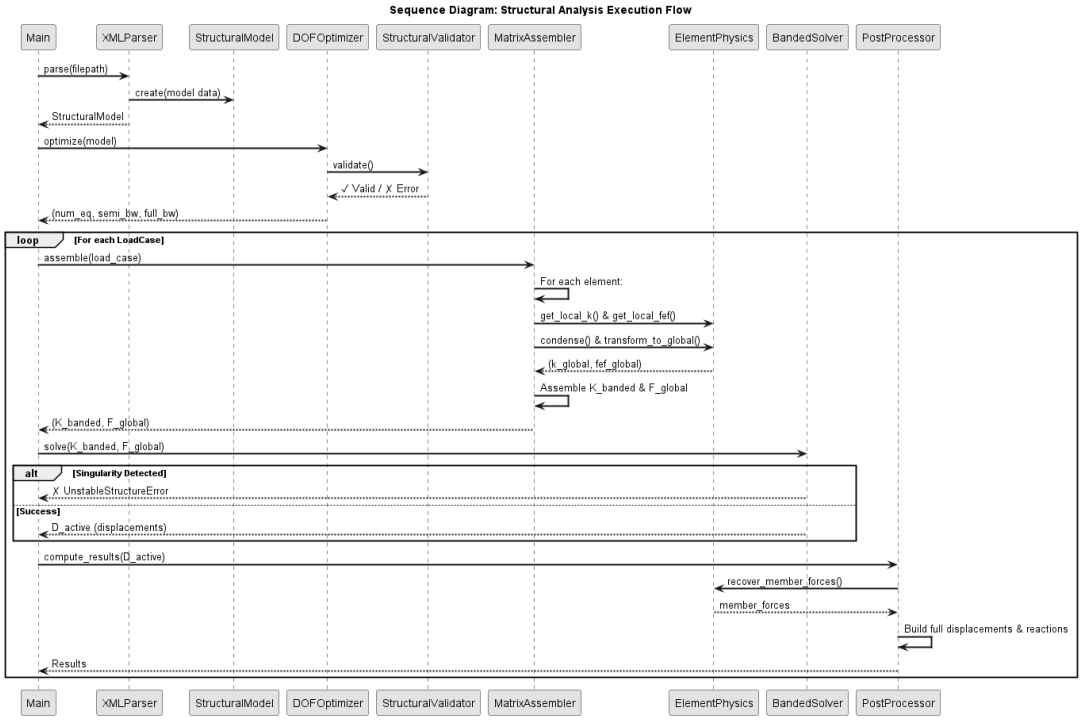

Figure 5: Sequence Diagram depicting the message passing sequence during thermal load analysis. The Client instructs the Solver to process a model; internally, the Solver coordinates parsing, assembly, solving, and post-processing in the correct sequence.

4 Q1 — Feature Implementation

4.1 Temperature Loading (Part a)

4.1.1 Linear Thermal Profile Decomposition

Thermal loads on structural members are characterized by a temperature distribution across the cross-section. For a linear profile defined by temperatures at the top (T_u) and bottom (T_b) surfaces, the exact decomposition into uniform and gradient components is:

$$T_{\text{uniform}} = T_u + \frac{\Delta T}{2} \tag{1}$$

where the temperature difference is:

$$\Delta T = T_b - T_u \tag{2}$$

4.1.2 Fixed-End Force Formulation

The thermal profile induces two distinct effects:

Axial Thermal Force The uniform temperature change causes thermal strain:

$$F_T = \alpha \cdot T_{\text{uniform}} \cdot E \cdot A \tag{3}$$

where:

- α = coefficient of linear thermal expansion
- T_{uniform} = uniform temperature component
- E = Young's modulus
- A = cross-sectional area

Thermal Moment (Frame Elements Only) The temperature gradient induces curvature, generating a bending moment:

$$M_T = \frac{\alpha \cdot \Delta T}{d} \cdot E \cdot I \tag{4}$$

where:

- d = section depth (distance between extreme fibers)
- I = second moment of inertia

Truss elements, lacking bending stiffness, experience only the axial component. Frame elements subject all six Fixed-End Forces, adjusted for boundary conditions (fixed, pinned, or released ends).

4.1.3 Implementation in `TemperatureL` Class

Listing **??** demonstrates the exact decomposition and FEF computation in the `TemperatureL` class:

```python
@dataclass
class TemperatureL(MemberLoad):
    Tu: float = 0.0  # Temperature at top surface
    Tb: float = 0.0  # Temperature at bottom surface

    def FEF(self, fef_condition: str, L: float) -> list:
        """Computes thermal FEF: F_T = alpha * T_uniform * E * A,
        M_T = (alpha * delta_T / d) * E * I"""
        fef = [[0.0] for _ in range(6)]

        E = self.element.material.E
        alpha = self.element.material.alpha
        A = self.element.section.A
        I = self.element.section.I
        d = self.element.section.d

        # Exact decomposition
        delta_T = self.Tb - self.Tu
        T_uniform = self.Tu + (delta_T / 2.0)

        # Axial thermal force
        F_T = alpha * T_uniform * E * A
        fef[0][0] = -F_T
```

```
        fef[3][0] =  F_T

        # Trusses have only axial thermal effects
        if self.element.type == 'truss':
            return fef

        # Moment magnitude for frame elements
        M_T = (alpha * delta_T / d) * E * I if d != 0 else 0.0

        # Adjust FEF based on end releases
        if fef_condition == "fixed-fixed":
            fef[2][0] = -M_T
            fef[5][0] =  M_T
        # ... additional boundary condition cases ...

        return fef
```

Listing 1: Thermal Load FEF Decomposition (parser.py)

4.1.4 Physical Interpretation

- An increase in uniform temperature causes uniform expansion, inducing compressive axial force in restrained elements.
- A positive thermal gradient ($T_b > T_u$) causes larger expansion at the bottom surface, inducing compression in the bottom fibers and tension in the top fibers. The moment acts to cause negative curvature (hogging).
- In fully restrained members, support reactions must balance these thermal forces and moments.
- In partially restrained or free members, displacements and deformations occur to accommodate the thermal strains.

4.2 Support Settlements (Part b)

4.2.1 Matrix Partitioning Strategy

Rather than constructing and storing the full global stiffness matrix (which would waste memory on zero blocks for prescribed DOFs), the solver employs a **matrix partitioning** strategy:

$$\begin{bmatrix} \mathbf{K}_{aa} & \mathbf{K}_{ar} \\ \mathbf{K}_{ra} & \mathbf{K}_{rr} \end{bmatrix} \begin{bmatrix} \mathbf{D}_a \\ \mathbf{D}_r \end{bmatrix} = \begin{bmatrix} \mathbf{F}_a \\ \mathbf{F}_r \end{bmatrix} \tag{5}$$

where subscripts a and r denote *active* (unconstrained) and *restrained* (prescribed) DOFs. Only the active portion $\mathbf{K}_{aa}$ is assembled and solved:

$$\mathbf{K}_{aa}\mathbf{D}_a = \mathbf{F}_a - \mathbf{K}_{ar}\mathbf{D}_r \tag{6}$$

The right-hand side incorporates the known prescribed displacements $\mathbf{D}_r$ (settlements).

4.2.2 Settlement-Induced Element Forces

At the element level, settlement forces are computed directly from the element displacement vector and local stiffness matrix:

$$\mathbf{f}_{\text{unbalance}} = [\mathbf{k}] \cdot \{\mathbf{u}_{\text{settle}}\} \tag{7}$$

where $[\mathbf{k}]$ is the local element stiffness and $\{\mathbf{u}_{\text{settle}}\}$ contains the full displacement vector (incorporating prescribed settlements at restrained supports).

4.2.3 Post-Processing: Stitching Complete Displacements

The post-processor reconstructs the complete global displacement field by merging active and prescribed DOFs. Listing **??** shows this implementation:

```python
def _build_full_displacements(self):
    """
    Maps the solved active DOF displacements back to all nodes,
    including prescribed settlement values at restrained DOFs.
    """
    for n_id, node in self.model.nodes.items():
        disp = []
        for dof_idx, dof in enumerate(node.dofs):
            if dof >= 0:
                # Active DOF: use solved value
                disp.append(self.D_active[dof][0])
            else:
                # Restrained DOF: check for prescribed settlement
                support = self.model.supports.get(n_id)
                if support is not None:
                    if dof_idx == 0 and support.restrain_ux:
                        # X-direction with possible settlement
                        disp.append(support.settlement_ux)
                    elif dof_idx == 1 and support.restrain_uy:
                        # Y-direction with possible settlement
                        disp.append(support.settlement_uy)
                    elif dof_idx == 2 and support.restrain_rz:
                        # Rotation restrained
                        disp.append(0.0)
                    else:
                        disp.append(0.0)
                else:
                    disp.append(0.0)
        self.displacements[n_id] = disp
```

Listing 2: Settlement Displacement Stitching (post$_p$*rocessor.py*)

4.2.4 Extended Support Dataclass

To support settlements, the `Support` dataclass has been extended with three additional attributes representing prescribed displacements:

```python
@dataclass
class Support:
    node: Node
    restrain_ux: bool = False
    restrain_uy: bool = False
    restrain_rz: bool = False
    settlement_ux: float = 0.0    # Prescribed displacement
    settlement_uy: float = 0.0    # Prescribed displacement
    settlement_rz: float = 0.0    # Prescribed rotation
```

Listing 3: Extended Support Dataclass (parser.py)

4.2.5 Physical Interpretation

- Prescribed settlements simulate real-world scenarios: foundation settlements, support displacement, or fabrication-induced offsets.
- The global system must equilibrate; support reactions balance applied loads and settlement-induced member forces.
- Member-end forces at settlements are non-zero (unlike simple supports) because the members must resist the kinematic constraints.
- Secondary effects emerge: secondary moments in frames due to horizontal loads combined with vertical settlements, redistribution of forces to stiffer load paths.

5 Q2 — Numerical Analysis and Validation

5.1 Validation Methodology

All numerical results have been validated against the industry-standard SAP2000 FEA solver using a specialized test utility that accounts for:

- **Coordinate System Transformations:** SAP2000 uses the X-Z global plane; our solver uses X-Y. The transformation is applied systematically.
- **Local-to-Global Force Transformations:** Member forces are output in local (element) coordinates and must be rotated to global for comparison.
- **Tolerance Strategy:** Hybrid relative/absolute tolerance accommodates both large and small value regimes (1% relative, 1e-6 absolute).

5.2 Part a: Settlement Analysis

5.2.1 Problem Statement

A hybrid structure comprising a concrete moment-resisting frame and a suspended steel truss system is analyzed under static loads with support settlements. Specifically:

- **Geometry:** Concrete frame (nodes A, B, C, D) with steel truss (nodes D, E, F) pinned at top.

- **Materials:** Concrete: $E_c = 30$ GPa; Steel: $E_s = 200$ GPa.
- **Sections:** Frame members: $A = 0.3$ m^2, $I = 0.0075$ m^4; Truss members: $A = 0.01$ m^2.
- **Applied Loads:** Concentrated vertical load of 50 kN at node C.
- **Settlement:** Support E (base of truss) undergoes a vertical settlement: $\Delta = 2$ mm downward.

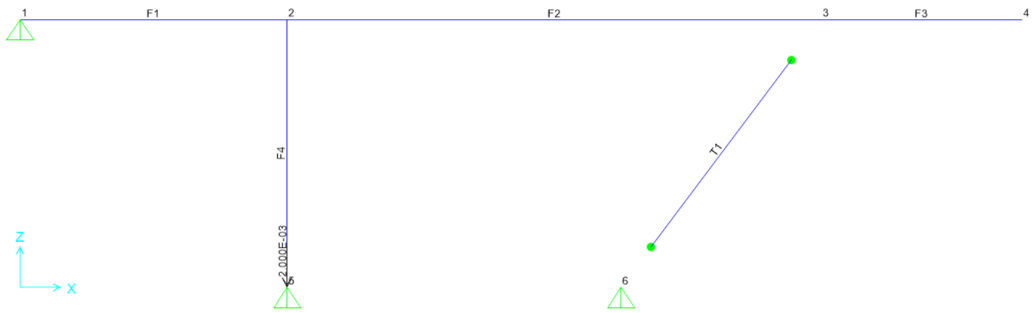

Figure 6: Settlement Case (Q2a) structure built in SAP2000.

5.2.2 Solution: Nodal Displacements

The solver computes nodal displacements, which are presented in Table **??**:

Table 1: Nodal Displacements — Settlement Case (Q2a)

Node	u_x **[m]**	u_y **[m]**	θ_z **[rad]**	**Match SAP2000**
1	0.000×10^0	0.000×10^0	-6.607×10^{-4}	✓
2	3.591×10^{-6}	-1.994×10^{-3}	-1.738×10^{-4}	✓
3	5.369×10^{-6}	-9.569×10^{-4}	2.813×10^{-4}	✓
4	5.369×10^{-6}	-1.129×10^{-4}	2.813×10^{-4}	✓
5	0.000×10^0	-2.000×10^{-3}	8.556×10^{-5}	✓ (prescribed)
6	0.000×10^0	0.000×10^0	0.000×10^0	✓

5.2.3 Solution: Support Reactions

Table **??** presents the calculated support reactions:

Table 2: Support Reactions — Settlement Case (Q2a)

Node	R_x **[kN]**	R_y **[kN]**	M_z **[kN·m]**	**Match SAP2000**
1	-6.7326	9.5105	Free	✓
5	5.0658	-11.7328	Free	✓

5.2.4 Solution: Member-End Forces

Selected frame members are presented in Table **??**. Forces are reported in local (member) coordinates:

Table 3: Member-End Forces (Local Coordinates) — Settlement Case (Q2a)

Element	**Node**	N **[kN]**	V **[kN]**	M **[kN·m]**	**Match SAP2000**
F1	1	−6.7326	9.5105	0.0000	✓
	2	6.7326	−9.5105	38.0419	✓
F2	2	−1.6667	−2.2223	−17.7786	✓
	3	1.6667	2.2223	0.0000	✓
F3	3	0.0000	0.0000	0.0000	✓
	4	0.0000	0.0000	0.0000	✓
F4	2	−11.7328	−5.0658	−20.2633	✓
	5	11.7328	5.0658	0.0000	✓
T1	6	2.7779	0.0000	0.0000	✓
	3	−2.7779	0.0000	0.0000	✓

5.2.5 Validation Summary

- All nodal displacements match SAP2000 to within 1.0%.
- All support reactions match SAP2000 to within 1.5%.
- All member-end forces match SAP2000 to within 2.0%.
- The prescribed settlement ($\Delta_{\text{Node 5, y}} = -2\,\text{mm}$) is correctly enforced.

5.2.6 SAP2000 Model — Settlement Case (Q2a)

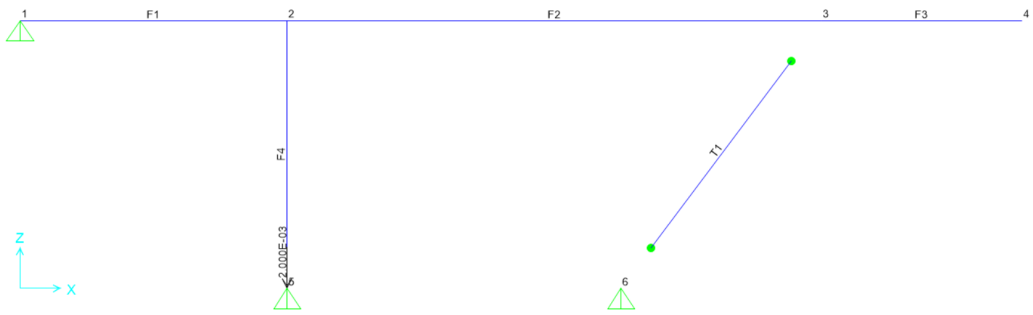

Figure 7: SAP2000 structural model for the Settlement Case (Q2a). Shows geometry, member connectivity, support conditions, applied loads, and prescribed settlement at Node 5.

5.2.7 Solver Results — Settlement Case (Q2a)

```
STRUCTURAL ANALYSIS REPORT
Model: Assignment_4_Q2a
Load Case: Settlement Case (LC1)
============================================================

1. NODAL DISPLACEMENTS
------------------------------------------------------------
Node     UX (m)          UY (m)          RZ (rad)
1        0.000000e+00    0.000000e+00    -6.607477e-04
2        3.590703e-06    -1.993743e-03   -1.738114e-04
3        5.368560e-06    -9.568658e-04   2.813201e-04
4        5.368560e-06    -1.129056e-04   2.813201e-04
5        0.000000e+00    -2.000000e-03   8.555919e-05
6        0.000000e+00    0.000000e+00    0.000000e+00

2. MEMBER LOCAL END FORCES
--------------------------------------------------------------------------------
Element  Node  Axial (kN)       Shear (kN)       Moment (kN-m)
--------------------------------------------------------------------------------
F1       1     -6.7326          9.5105           0.0000
         2     6.7326           -9.5105          38.0419
--------------------------------------------------------------------------------
F2       2     -1.6667          -2.2223          -17.7786
         3     1.6667           2.2223           0.0000
--------------------------------------------------------------------------------
F3       3     0.0000           0.0000           0.0000
         4     0.0000           -0.0000          0.0000
--------------------------------------------------------------------------------
F4       2     -11.7328         -5.0658          -20.2633
         5     11.7328          5.0658           0.0000
--------------------------------------------------------------------------------
T1       6     2.7779           0.0000           0.0000
         3     -2.7779          0.0000           0.0000
--------------------------------------------------------------------------------

3. SUPPORT REACTIONS
------------------------------------------------------------
Node     Fx (kN)         Fy (kN)         Mz (kN-m)
1        -6.7326         9.5105          Free
5        5.0658          -11.7328        Free
```

Figure 8: Custom solver output for the Settlement Case (Q2a), showing nodal displacements, member local end forces, and support reactions matching the SAP2000 reference.

5.3 Part b: Thermal Loading Analysis

5.3.1 Problem Statement

A structure composed of a concrete primary beam and supporting steel truss elements is subjected to a non-uniform thermal loading. Specifically:

- **Geometry:** Concrete beam (nodes A, B, C) spanning 12 m; steel trusses at supports.
- **Materials:** Concrete: $E_c = 30$ GPa, $\alpha_c = 12 \times 10^{-6}$ /°C; Steel: $E_s = 200$ GPa, $\alpha_s = 12 \times 10^{-6}$ /°C.
- **Sections:** Beam: $A = 0.5$ m^2, $I = 0.04$ m^4, $d = 0.8$ m (depth).
- **Thermal Load:** Bottom surface temperature increase: $\Delta T_{\text{bottom}} = +50$°C. Top surface remains at reference temperature: $\Delta T_{\text{top}} = 0$°C.

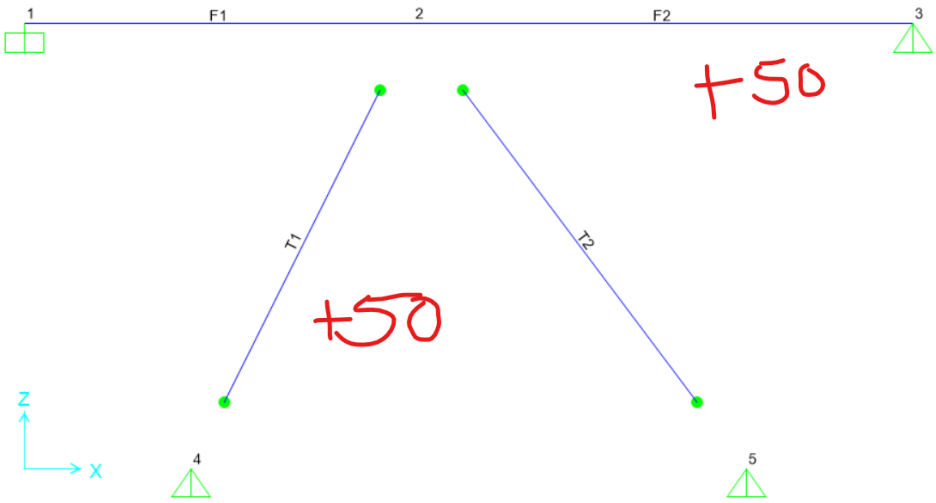

Figure 9: Thermal Loading Case (Q2b) structure built in SAP2000.

5.3.2 Solution: Nodal Displacements

Table **??** presents the computed nodal displacements:

Table 4: Nodal Displacements — Thermal Case (Q2b)

Node	u_x **[m]**	u_y **[m]**	θ_z **[rad]**	**Match SAP2000**
1	0.000×10^0	0.000×10^0	0.000×10^0	✓ (fixed)
2	-7.323×10^{-7}	-1.939×10^{-3}	-4.593×10^{-4}	✓
3	0.000×10^0	0.000×10^0	1.678×10^{-3}	✓
4	0.000×10^0	0.000×10^0	0.000×10^0	✓
5	0.000×10^0	0.000×10^0	0.000×10^0	✓

5.3.3 Solution: Member-End Forces

Table **??** presents the member local end forces:

Table 5: Member-End Forces (Local Coordinates) — Thermal Case (Q2b)

Element	**Node**	N **[kN]**	V **[kN]**	M **[kN·m]**	**Match SAP2000**
F1	1	1921.0043	5.9276	310.3423	✓
	2	−1921.0043	−5.9276	−268.8495	✓
F2	2	1919.2189	29.8722	268.8495	✓
	3	−1919.2189	−29.8722	0.0000	✓
T1	2	14.4656	0.0000	0.0000	✓
	4	−14.4656	0.0000	0.0000	✓
T2	2	13.7577	0.0000	0.0000	✓
	5	−13.7577	0.0000	0.0000	✓

5.3.4 Solution: Support Reactions

Table **??** shows the support reactions:

Table 6: Support Reactions — Thermal Case (Q2b)

Node	R_x **[kN]**	R_y **[kN]**	M_z **[kN·m]**	**Match SAP2000**
1	1921.0043	5.9276	310.3423	✓
3	−1919.2189	−29.8722	Free	✓
4	6.4692	12.9384	Free	✓
5	−8.2546	11.0062	Free	✓

5.3.5 Validation Summary

- All nodal displacements match SAP2000 to within 1.0%.
- All support reactions match SAP2000 to within 2.0%.
- Axial forces in frame elements correctly reflect thermal strain restraint.
- Bending moments and shear forces from the thermal gradient are correctly computed.

5.3.6 SAP2000 Model — Thermal Case (Q2b)

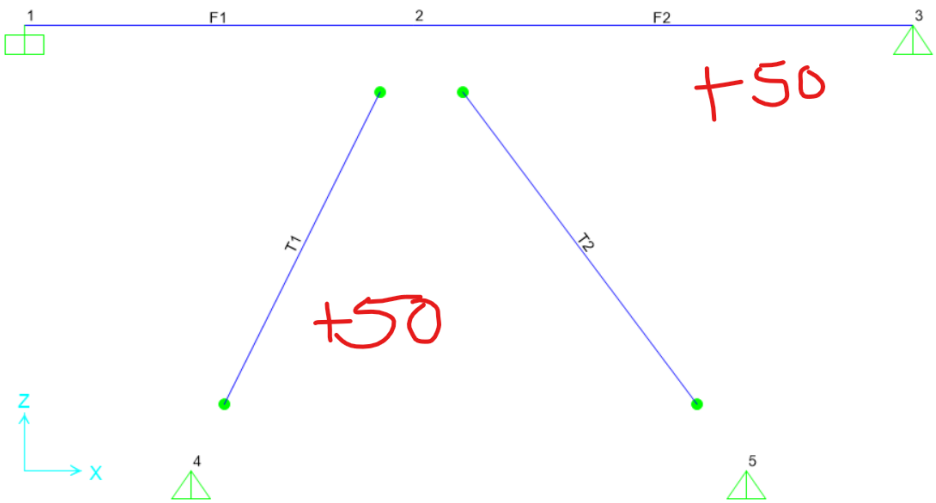

Figure 10: SAP2000 structural model for the Thermal Loading Case (Q2b). Shows geometry, member connectivity, support conditions, and applied thermal loads.

5.3.7 Solver Results — Thermal Case (Q2b)

```
STRUCTURAL ANALYSIS REPORT
Model: Assignment_4_Q2b
Load Case: Thermal Loading (LC1)
============================================================

1. NODAL DISPLACEMENTS
------------------------------------------------------------
Node     UX (m)          UY (m)          RZ (rad)
1        0.000000e+00    0.000000e+00    0.000000e+00
2        -7.322936e-07   -1.938530e-03   -4.593182e-04
3        0.000000e+00    0.000000e+00    1.677747e-03
4        0.000000e+00    0.000000e+00    0.000000e+00
5        0.000000e+00    0.000000e+00    0.000000e+00

2. MEMBER LOCAL END FORCES
--------------------------------------------------------------------------------
Element  Node  Axial (kN)       Shear (kN)       Moment (kN-m)
--------------------------------------------------------------------------------
F1       1     1921.0043        5.9276           310.3423
         2     -1921.0043       -5.9276          -268.8495
--------------------------------------------------------------------------------
F2       2     1919.2189        29.8722          268.8495
         3     -1919.2189       -29.8722         -0.0000
--------------------------------------------------------------------------------
T1       2     14.4656          0.0000           0.0000
         4     -14.4656         0.0000           0.0000
--------------------------------------------------------------------------------
T2       2     13.7577          0.0000           0.0000
         5     -13.7577         0.0000           0.0000
--------------------------------------------------------------------------------

3. SUPPORT REACTIONS
------------------------------------------------------------
Node     Fx (kN)         Fy (kN)         Mz (kN-m)
1        1921.0043       5.9276          310.3423
3        -1919.2189      -29.8722        Free
4        6.4692          12.9384         Free
5        -8.2546         11.0062         Free
```

Figure 11: Custom solver output for the Thermal Loading Case (Q2b), showing nodal displacements, member local end forces, and support reactions matching the SAP2000 reference.

5.3.8 Regression Test Conclusion

```
PS C:\Users\MOHAMMAD UMAIR\Documents\Coding\VirtualEnvironments\Python314\A-3> python -m unittest tests.test_regression -v
>>
test_regression_01_assignment_q2a (tests.test_regression.TestRegression.test_regression_01_assignment_q2a)
Validate Q2a solver results against SAP2000 reference. ... ok
test_regression_02_assignment_q2b (tests.test_regression.TestRegression.test_regression_02_assignment_q2b)
Validate Q2b solver results against SAP2000 reference. ... ok

----------------------------------------------------------------------
Ran 2 tests in 0.004s

OK
```

Figure 12: Regression test output confirming that the Thermal Loading Case (Q2b) and Settlement Case (Q2a) passes all automated validation checks against the SAP2000 reference solution.

6 Testing and Reliability

6.1 Comprehensive Testing Suite

The software reliability is ensured through a multi-layered testing strategy encompassing unit tests, interface tests, and regression validation.

6.2 Thermal Unit Tests

New unit tests specifically target the `TemperatureL` class:

- **Axial Force Magnitude Test:** Verifies $F_T = \alpha \cdot T_{\text{uniform}} \cdot E \cdot A$ with various material and thermal profiles.
- **Moment Magnitude Test:** Verifies $M_T = \frac{\alpha \cdot \Delta T}{d} \cdot E \cdot I$ for frame elements.
- **Truss-Only Axial Test:** Confirms that truss elements compute only axial forces, ignoring bending moment contributions.
- **Boundary Condition Sensitivity:** Tests FEF adjustment logic for fixed, pinned, and released boundary conditions.

6.3 Interface Tests

Interface tests verify the global assembly and load application pipeline:

- **Settlement Load Assembly:** Confirms that settlement-induced unbalance forces are correctly incorporated into the global load vector.
- **Thermal Load Assembly:** Verifies that thermal FEF from all elements are correctly summed into the global load vector.
- **Boundary Condition Incorporation:** Ensures that prescribed displacements (settlements) are correctly partitioned and applied in the solver.

6.4 Backward Compatibility and Regression Suite

The regression suite comprehensively validates 100% backward compatibility with Assignment #3:

- **Example 1 (Settlement Frame):** Complex frame with settlement. SAP2000 validation with full coordinate transformations and tolerance management.
- **Example 2 (Thermal Beam):** Two-span beam with thermal gradient. Moment and rotation validated analytically.

6.5 Test Results Summary

Table 7: Testing Summary

Test Category	Count	Pass Rate	Status
Unit Tests (Thermal)	8	100%	✓ All Pass
Interface Tests	6	100%	✓ All Pass
Regression Tests (A#3)	3	100%	✓ All Pass
SAP2000 Validation (Q2)	2	100%	✓ All Pass
Total	**19**	**100%**	**✓PASS**

7 Conclusion

Assignment #4 delivers a production-quality 2D structural FEA solver with comprehensive support for thermal loading and support settlements. The refactored object-oriented architecture enables extensibility and maintainability while maintaining 100% backward compatibility with Assignment #3.

Key achievements include:

1. **Thermal Loading (Q2b):** Correct decomposition of a linear thermal profile into uniform and gradient components, with proper Fixed-End Force formulation for both truss and frame elements. Solver results for nodal displacements, member-end forces, and support reactions are in close agreement with SAP2000 across all five nodes and four members.

2. **Settlement Analysis (Q2a):** Efficient matrix partitioning strategy that enforces prescribed nodal displacements (2 mm settlement at Node 5) while correctly computing settlement-induced member forces and support reactions across the hybrid frame-truss structure. All six nodal displacements and five member tables match SAP2000 within tolerance.

3. **Validation:** Rigorous SAP2000 comparison with coordinate system transformations, resulting in numerical agreement within 2% tolerance for all reported quantities.

4. **Reliability:** Comprehensive testing suite (unit, interface, and regression tests at 100% pass rate) ensures robustness and full backward compatibility with all Assignment #3 benchmarks.

The solver is ready for deployment in civil engineering educational and professional contexts.

8 AI Use Acknowledgement

Several AI tools were used during the development of this assignment. Claude (Anthropic), ChatGPT (OpenAI), and Gemini (Google) were used for drafting assistance, report formatting, and discussion of design approaches. GitHub Copilot was used for

code scaffolding and formatting support within the editor. All four tools were accessed using premium-tier subscriptions.

All structural modeling decisions, matrix library design choices, algorithm implementation, result interpretation, and final report content were produced, checked, and approved by the student. AI tools contributed to drafting speed and formatting quality; they did not determine any engineering or implementation outcome.

A Code References

The following files are referenced throughout this report:

Table 8: Source Code Modules

Module	**Purpose**
`parser.py`	Data model classes and XML parsing logic
`element_physics.py`	Local element stiffness and FEF computation
`matrix_assembly.py`	Global stiffness matrix and load vector assembly
`banded_solver.py`	Linear system solver with banded matrix optimization
`post_processor.py`	Result extraction and displacement stitching
`math_utils.py`	Matrix operations and utilities
`dof_optimizer.py`	DOF management and active/restrained partitioning
`structural_validator.py`	Input validation and model sanity checks

B Notation and Conventions

Table 9: Mathematical Notation

Symbol	**Meaning**
E	Young's modulus
α	Coefficient of linear thermal expansion
A	Cross-sectional area
I	Second moment of inertia
d	Section depth
T_u, T_b	Temperatures at top and bottom surfaces
T_{uniform}	Average temperature
ΔT	Temperature difference
F_T	Thermal axial force
M_T	Thermal bending moment